

ToSh Cheat Sheet: Filesystem And File I/O

Navigation, discovery, mutation, text/binary commands, and managed handles

command reference + tests

Navigation

Command	Use
pwd	current directory as an object
cd path	change directory
dirs	inspect the directory stack
back	move backward in the stack
forward	move forward in the stack

```
pwd
cd src
dirs
back
forward
```

Discovery

```
ls -la
ls -R --group-directories-first
stat README.md
tree -L 2
```

```
find . -name *.tosh
find . -type dir -maxdepth 2
glob **/*.tosh
realpath ./README.md
readlink ./some-link
```

Predicate	Meaning
exists	path exists
is-file	regular file check
is-dir	directory check
is-link	symlink check
basename	file name only
dirname	parent path only

Typed File Rows

```
ls -la | where _.Type == file | get { Name, Size, Modified }
find . -name *.cs | first 10 | get FullName
stat README.md | inspect
```

Filesystem commands emit typed entries, so fields like **Name**, **Type**, **Size**, and **Modified** can be sorted or filtered directly.

Mutation

```
mkdir scratch
touch scratch/notes.txt
cp README.md scratch/readme-copy.md
mv scratch/readme-copy.md scratch/info.md
rm scratch/info.md
```

```
chmod 755 ./script
ln -s target.txt link.txt
mkdir-temp
tempfile
```

Command	Notes
mkdir	supports parent creation flags
touch	create or update timestamps
cp, mv, rm	typed file operations
chmod, chown, ln	Unix-style metadata/link tools

Read And Write Files

```
cat README.md
read-file README.md
read-lines README.md | first 5
read-bytes icon.png
```

```
write-file scratch/notes.txt "hello"
append-file scratch/notes.txt "world"
write-bytes scratch/blob.bin [1, 2, 3]
```

Command	Shape
read-file	one file -> one string
read-lines	lines as separate values
read-bytes	byte arrays
lines /	text stream conversion
join-lines	
as-file	materialize pipeline output as a temp file

Managed Handles

```
var reader = (open-file --read scratch/notes.txt)
read-line-from $reader
read-to-end $reader
close $reader
```

```
var writer = (open-file --write scratch/log.txt)
write-line-to $writer "ready"
flush $writer
close $writer
```

Handle Toolbox

Command	Use
open-file	create text or binary file handles
read-from	read a chunk from a handle
read-line-from	read the next text line
read-to-end	consume the rest of the handle
write-to /	write bytes or text
write-line-to	
seek, position, length	stream positioning
copy-to	copy one handle into another
flush, close	finish safely

Useful Recipes

```
# biggest files here
ls -la | where _.Type == file | sort Size | reverse | first 10
```

```
# recent logs
find . -name *.log | where _.Modified > ((date now) - 1d)
```

```
# line-oriented pipeline
read-lines README.md | grep "ToSh" | first 5
```

```
# stage pipeline output into a file for externals
var names = (ls | get Name | sort | as-file text)
/bin/cat $names
```

External-System Wrappers

```
df
du .
lsblk -l
findmnt
systemctl --type service | first 5
ip addr
```

- Many system commands are wrapped and return structured rows instead of raw text.
- That means **sort**, **where**, and **summarize** work on their fields directly.

Good habit: prefer ToSh wrappers like **ls**, **find**, **df**, **ip**, and **systemctl** when you want typed data; drop to native tools when you need exact external behavior.

ToSh Cheat Sheet: Filesystem And File I/O

Typed wrappers, stream positioning, and path-heavy recipes

Disk And Mount Views

```
df --total
df -t ext4 --show FileSystem,Type,UsePercent,MountedOn
df . | get { FileSystem, MountedOn }

du -s .
du -a -c --time
du -x ./projects | get { Name, Size, Modified }
```

df and du return typed size fields, so sorting and summarizing disk usage stays object-oriented instead of text-oriented.

Structured System Wrappers

```
findmnt -l | where _.Target.StartsWith("/run")
lsblk -l -f | where _.FsType == "ntfs"
lsfd --summary=only

ip addr | each { _.Addresses } | flatten | get Address
systemctl status sshd.service | get { Id, ActiveState, MainPID,
    RecentLogCount }
journalctl -u sshd.service | first 10
```

Path Utilities

```
glob -a **/*.cs
realpath ./relative/path
readlink -f ./chain
basename /home/user/file.txt
dirname /home/user/file.txt
```

Tool	When to use it
glob	shell-style wildcard expansion into typed paths
realpath	canonicalize a relative path
readlink	inspect a symlink target or whole chain
basename /	split path text safely
dirname	

Handle Positioning

```
var h = (open-file --binary ./data.bin)
length $h
position $h
seek $h 128 current
read-from $h 32

var src = (open-file --read ./a.txt)
var dst = (open-file --write ./b.txt)
copy-to $src $dst
close $src $dst
```

seek returns the handle again, so you can keep piping through read-from, read-to-end, or copy-to.

Copy, Link, And Mode Tricks

```
cp -u notes.txt backup.txt
cp -s file.txt symlink.txt
cp -l file.txt hardlink.txt
chmod +x script.sh
chmod u+rw,go-w file.txt
```

Write Strategies

```
echo alpha beta | write-file ./notes.txt
echo tail line | append-file ./notes.txt
read-bytes ./source.bin | write-bytes ./copy.bin

var log = (open-file --append ./run.log)
write-line-to $log "ready"
flush $log
close $log
```

Practical File Recipes

```
find . -name *.log | where _.Size >= 10mb | get { Name, Size }
glob **/*.md | where _.Modified > ((date now) - 7d)
ls -la | where _.Type == file | sort Modified | reverse | first 20

var tmp = (mkdir-temp report)
var out = (tempfile export .csv)
$rows | to csv | write-file $out
realpath $out
```

Mental Model

- Path-first wrappers like ls, stat, find, df, and lsblk try hard to keep you in typed objects.
- Managed handles are explicit on purpose: you can inspect them, pass them around, and close them deterministically.
- Reach for external programs when you need exact native semantics; reach for ToSh wrappers when you want composable shell objects.